ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Е.О. Шумова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №8

«Описание классов и порождение объектов»

по курсу: ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2024

Задание

1. Выбранная тема - парковки.

2. Проанализировать предметную область, выделить сущности предметной области и сформулировать разрабатываемый функционал. В предметной области должна быть реализована многозначная зависимость.

3. На основе сущностей предметной области спроектировать иерархии (или совокупности) связанных классов. Для каждого класса в этом разделе должны быть указаны имена полей (с указанием семантического смысла для каждого поля) и перечислены методы класса (т.е. хедеры классов).

4. В работе рекомендуется использовать не менее двух паттернов. Паттерны должны быть разных типов: порождающие (кроме Сингелтона), поведенческие, структурные. Студент должен обосновать использование тех или иных паттернов. Результатом проектирования является представленная диаграмма классов (или несколько диаграмм), а также должна быть диаграмма классов для каждого используемого паттерна (с именами классов студента)

5. Собственно, разработка программы. На основе перечисленного функционала (из пункта 2) формируете структуру меню и уделяете внимание каждому пункту.

- работа выполняется в графическом приложении Qt

- используются паттерны (не менее 2-х)

- для ввода информации пользователем предусматриваются формы для ввода

- вывод информации (для выбора) осуществляется в виде таблицы (виджет QTableView)

- реализована многозначная зависимость

Сущности предметной области

1. MainWindow (Главное окно): Это основное окно приложения, где пользователь взаимодействует с интерфейсом. Здесь отображается информация о парковках, посетителях и билетах, а также доступны функции добавления и удаления записей.
2. Parking (Парковка): Класс, представляющий отдельную парковку. Содержит информацию о ее идентификаторе, местоположении (GPS), размере, типе, стоимости и статусе (занята/свободна).
3. Visitor (Посетитель): Этот класс представляет собой отдельного посетителя парковки. Хранит информацию о номере посетителя, имени, адресе и номере телефона.
4. Ticket (Билет): Класс, представляющий отдельный билет на парковку. Содержит информацию о парковке (идентификатор), посетителе (номер), и дате выдачи.
5. ParkingFactory (Фабрика парковок): Этот класс является фабрикой для создания объектов класса Parking. Он содержит методы для создания новых парковок и управления ими.
6. Visitors (Посетители): Класс для управления списком посетителей парковки. Содержит методы для добавления, удаления и загрузки посетителей из файла.
7. TicketManager (Менеджер билетов): Класс для управления билетами на парковку. Содержит методы для добавления и удаления билетов, а также для обновления статуса парковки при выдаче или возврате билета.

Разработанный функционал

1. Добавление парковки:
   - Пользователь заполняет поля для ввода данных о новой парковке, такие как идентификатор, местоположение (GPS), размер, тип и стоимость.
   - По нажатию кнопки "Добавить парковку" происходит создание новой парковки на основе введенных данных.
   - Созданная парковка добавляется в список доступных парковок и отображается в таблице парковок.
2. Удаление парковки:
   - Пользователь выбирает парковку из списка или вводит ее идентификатор.
   - По нажатию кнопки "Удалить парковку" происходит удаление выбранной парковки из списка.
   - Если парковка была свободна (статус "Имеется в наличии"), она удаляется без дополнительных проверок.
   - Если парковка занята (статус "Занято"), выводится предупреждение о невозможности удалить занятую парковку.
3. Добавление посетителя:
   - Пользователь вводит данные нового посетителя, такие как номер, имя, адрес и номер телефона.
   - По нажатию кнопки "Добавить посетителя" происходит создание нового посетителя на основе введенных данных.
   - Созданный посетитель добавляется в список посетителей и отображается в таблице.
4. Удаление посетителя:
   - Пользователь выбирает посетителя из списка или вводит его номер.
   - По нажатию кнопки "Удалить посетителя" происходит удаление выбранного посетителя из списка.
   - Если на посетителя есть билеты, выводится предупреждение о невозможности удалить посетителя.
5. Добавление билета:
   - Пользователь выбирает парковку из списка и вводит номер посетителя.
   - По нажатию кнопки "Добавить билет" создается новый билет на парковку для выбранного посетителя.
   - Статус выбранной парковки обновляется на "Занято".
6. Удаление билета:
   - Пользователь выбирает билет из списка или вводит номер парковки и номер посетителя.

- По нажатию кнопки "Удалить билет" происходит удаление выбранного билета из списка.
- Если билет успешно удален, статус парковки обновляется на "Имеется в наличии".

Хедеры классов (h-файлы). Для каждого поля указать семантический смысл

mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QFile>
#include <QTextStream>
#include "parkingfactory.h"
#include "visitors.h"
#include "ticketmanager.h" // Включаем заголовок TicketManager

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void addParking();
    void deleteParking();
```

```
    void updateTable();
    void loadParkingsToTable();
    void addVisitors();
    void deleteVisitors();
    void updateVisitorsTable();
    void addTicket(); // Добавляем слот для кнопки выдачи билетов
    void deleteTicket();
    void updateTicketTable();

private:
    Ui::MainWindow *ui; // Интерфейс главного окна
    ParkingFactory *parkingFactory; // Фабрика парковок
    Visitors *visitorsFactory; // Фабрика посетителей
    TicketManager *ticketManager; // Менеджер билетов
};

#endif
```

parkingfactory.h

```
#ifndef PARKINGFACTORY_H
#define PARKINGFACTORY_H

#include <QObject>
#include <QTableWidget>

class ParkingFactory : public QObject
{
    Q_OBJECT
```

```
public:
    explicit ParkingFactory(QObject *parent = nullptr);
    ~ParkingFactory();

    void addParking(const QString &id, const QString &gps, const QString &size, const QString &type, const QString &cost);
    void loadParkingsToTable(QTableWidget *table);

signals:
    void parkingAdded(); // Сигнал, отправляемый после добавления новой парковки

private:
    QStringList parkingData; // Данные о парковке для последующей загрузки в таблицу
};

#endif
```

ticketmanager.h

```
#ifndef TICKETMANAGER_H
#define TICKETMANAGER_H

#include <QObject>
#include <QFile>
#include <QTextStream>
#include <QMessageBox>
#include <QDateTime>
#include <QTableWidget>
```

```
class TicketManager : public QObject
{
    Q_OBJECT
public:
    explicit TicketManager(QObject *parent = nullptr);


signals:


public slots:
    void addTicket(const QString &parkingId, const QString &visitorNumber, const QString &date, QTableWidget *table);
    void updateTicketTable(QTableWidget *table); // Обновление таблицы билетов


};


#endif
```

visitors.h

```
#ifndef VISITORS_H
#define VISITORS_H

#include <QObject>
#include <QFile>
#include <QTextStream>
#include <QTableWidget>

class Visitors : public QObject
{
```

```
    Q_OBJECT
public:
    explicit Visitors(QObject *parent = nullptr);

    void addVisitor(const QString &number, const QString &name, const QString &address, const QString &phone);
    void saveVisitorsToFile(); // Сохранение посетителей в файл
    void loadVisitorsFromfile(QTableWidget *table); // Загрузка посетителей из файла в таблицу

signals:
    void visitorsAdded(); // Сигнал, отправляемый после добавления нового посетителя

private:
    QList<QStringList> visitorData; // Данные о посетителях
};

#endif
```

Диаграммы классов для каждого паттерна в нотации UML и общая диаграмма, описывающая проект приложения

Общая диаграмма классов:

```
@startuml
class MainWindow {
    - ui: Ui::MainWindow
    - parkingFactory: ParkingFactory
    - visitorsFactory: Visitors
    - ticketManager: TicketManager
    + addParking()
    + deleteParking()
    + updateTable()
    + loadParkingsToTable()
    + addVisitors()
    + deleteVisitors()
    + updateVisitorsTable()
    + addTicket()
    + deleteTicket()
    + updateTicketTable()
}

class ParkingFactory {
    + addParking(id: QString, gps: QString, size: QString, type: QString, cost: QString)
    + loadParkingsToTable(table: QTableWidget)
}

class Visitors {
    + addVisitor(number: QString, name: QString, address: QString, phone: QString)
```

```
    + saveVisitorsToFile()
    + loadVisitorsFromfile(table: QTableWidget)
}

class TicketManager {
    + addTicket(parkingId: QString, visitorNumber: QString, date: QString, table: QTableWidget)
    + updateTicketTable(table: QTableWidget)
}

class QTableWidget

MainWindow --> ParkingFactory
MainWindow --> Visitors
MainWindow --> TicketManager
ParkingFactory --> QTableWidget
Visitors --> QTableWidget
TicketManager --> QTableWidget
@enduml
```

Диаграмма “Фабричного метода”:

@startuml

class ParkingFactory {

- parkingData: QStringList

+ addParking(id: QString, gps: QString, size: QString, type: QString, cost: QString)

+ loadParkingsToTable(table: QTableWidget)

}

class TicketManager {

+ addTicket(parkingId: QString, visitorNumber: QString, date: QString, table: QTableWidget)

+ updateTicketTable(table: QTableWidget)

}

class Visitors {

+ addVisitor(number: QString, name: QString, address: QString, phone: QString)

+ saveVisitorsToFile()

+ loadVisitorsFromfile(table: QTableWidget)

}

class QTableWidget

ParkingFactory --> QTableWidget

@enduml

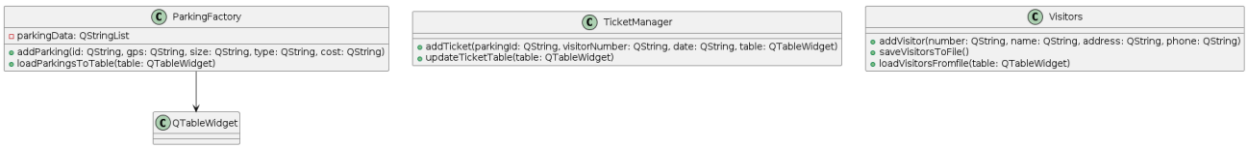


Диаграмма для паттерна “Наблюдатель”:

```
@startuml
class TicketManager {
    - observers: list<Observer>
    + attach(observer: Observer)
    + detach(observer: Observer)
    + notify()
}

class ConcreteObserver {
    + update()
}

class ConcreteSubject {
    - observers: list<Observer>
    + attach(observer: Observer)
    + detach(observer: Observer)
    + notify()
}

interface Observer {
    + update()
}

interface Subject {
    + attach(observer: Observer)
    + detach(observer: Observer)
    + notify()
}
```

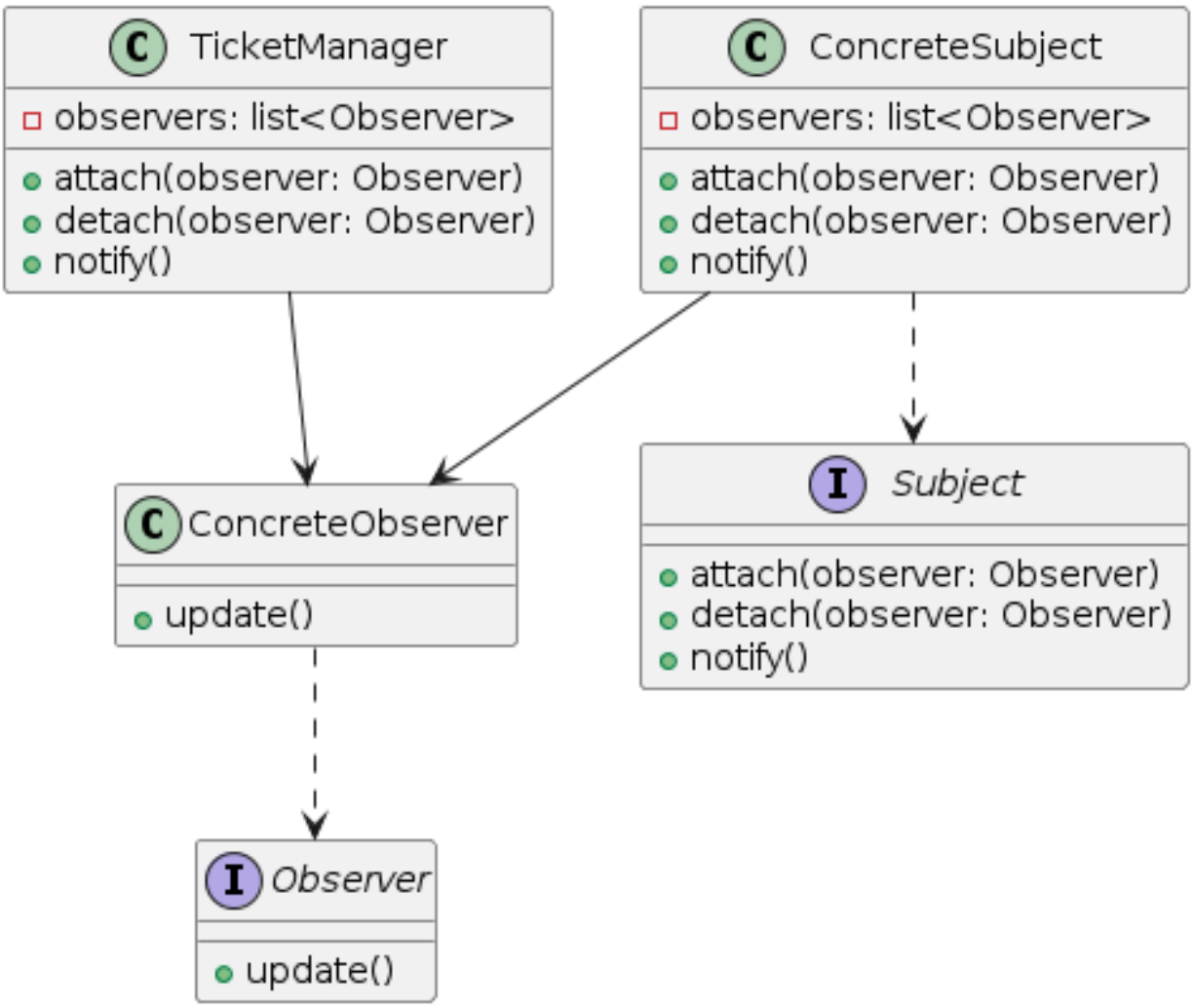
TicketManager --> ConcreteObserver

ConcreteSubject --> ConcreteObserver

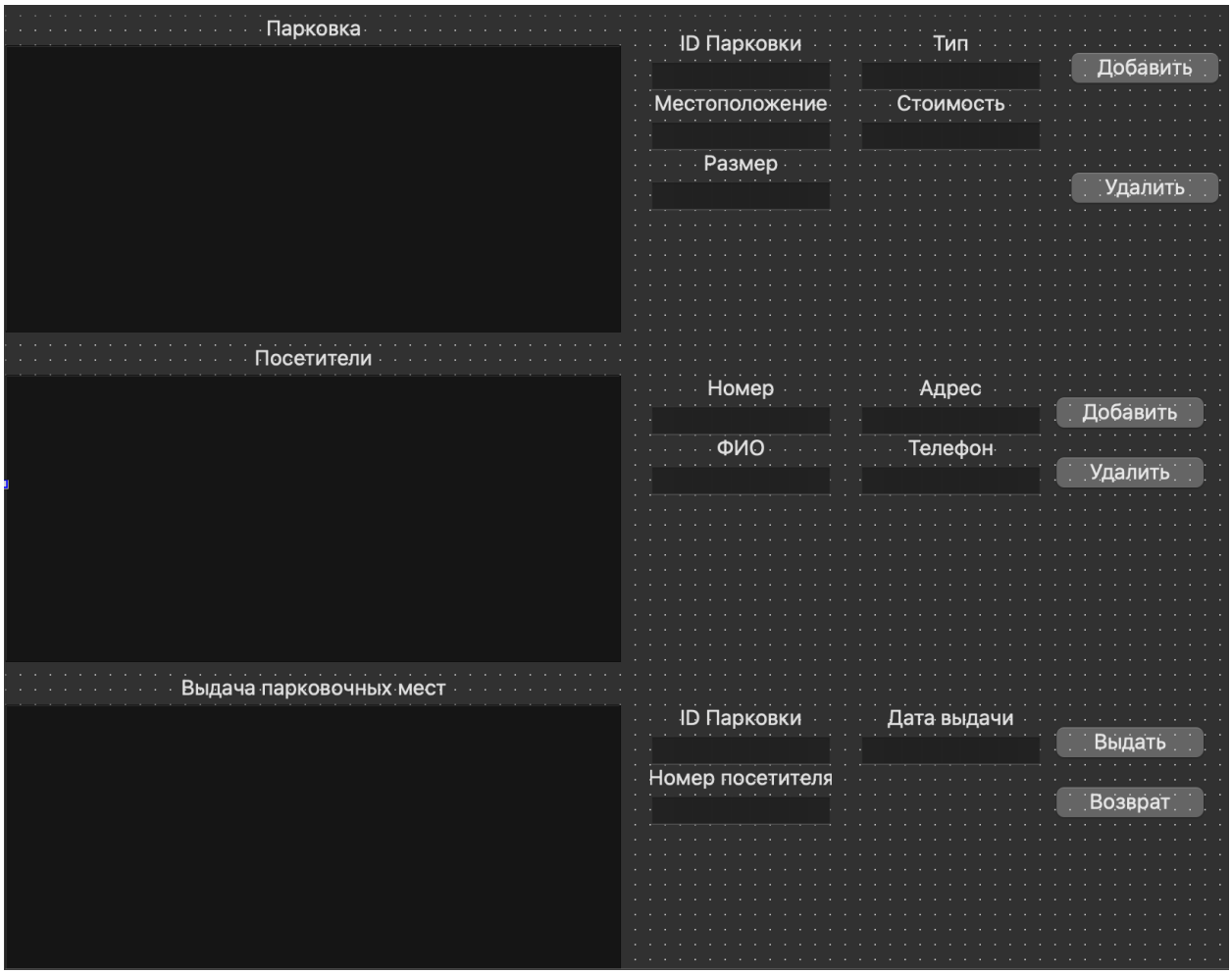
ConcreteSubject ..> Subject

ConcreteObserver ..> Observer

@enduml



Скриншоты всех разработанных программных форм в режиме дизайнера с указанием программных имен всех виджетов



1. tableParking - это объект класса QTableWidget, который используется для отображения данных о парковках.
2. tableVisitors - объект класса QTableWidget, предназначенный для отображения данных о посетителях.
3. tableTicket - объект класса QTableWidget, используемый для отображения данных о выданных билетах.
4. lineEdit_id_parking - это объект класса QLineEdit, предназначенный для ввода ID парковки.
5. lineEdit_gps - объект класса QLineEdit, используемый для ввода GPS-координат.
6. lineEdit_size - объект класса QLineEdit, используемый для ввода размера парковки.
7. lineEdit_type - объект класса QLineEdit, используемый для ввода типа парковки.
8. lineEdit_cost - объект класса QLineEdit, используемый для ввода стоимости парковки.

9. lineEdit_visitors_number - объект класса QLineEdit, предназначенный для ввода номера посетителя.
10. lineEdit_visitors_name - объект класса QLineEdit, используемый для ввода имени посетителя.
11. lineEdit_visitors_phone - объект класса QLineEdit, используемый для ввода номера телефона посетителя.
12. lineEdit_visitors_adress - объект класса QLineEdit, используемый для ввода адреса посетителя.
13. lineEdit_parking_number - объект класса QLineEdit, используемый для ввода номера парковки при выдаче билета.
14. lineEdit_visitors_number_t - объект класса QLineEdit, используемый для ввода номера посетителя при выдаче билета.
15. lineEdit_date - объект класса QLineEdit, используемый для ввода даты выдачи билета.
16. pushButton_add_parking - кнопка для добавления информации о парковке.
17. pushButton_delete_parking - кнопка для удаления информации о парковке.
18. pushButton_add_visitors - кнопка для добавления информации о посетителе.
19. pushButton_delete_visitors - кнопка для удаления информации о посетителе.
20. pushButton_add_ticket - кнопка для выдачи парковочного билета.
21. pushButton_delete_ticket - кнопка для возврата парковочного билета.

Листинг программы с комментариями

main.cpp

```
#include "mainwindow.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return a.exec();
}
```

mainwindow.cpp

```
#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    parkingFactory = new ParkingFactory(this); // Создаем объект ParkingFactory
    visitorsFactory = new Visitors(this); // Создаем объект Visitors
    ticketManager = new TicketManager(this);
```

```
    connect(parkingFactory, &ParkingFactory::parkingAdded, this, &MainWindow::updateTable); // Связываем сигнал о добавлении парковки с обновлением таблицы
    connect(visitorsFactory, &Visitors::visitorsAdded, this, &MainWindow::updateVisitorsTable); // Связываем сигнал о добавлении посетителя с обновлением таблицы

    // Обновляем таблицы при старте программы
    updateTable();
    updateVisitorsTable();
    updateTicketTable();

    connect(ui->pushButton_add_parking, &QPushButton::clicked, this, &MainWindow::addParking);
    connect(ui->pushButton_delete_parking, &QPushButton::clicked, this, &MainWindow::deleteParking);
    connect(ui->pushButton_add_visitors, &QPushButton::clicked, this, &MainWindow::addVisitors); // Связываем кнопку добавления посетителя с соответствующим слотом
    connect(ui->pushButton_delete_visitors, &QPushButton::clicked, this, &MainWindow::deleteVisitors);
    connect(ui->pushButton_add_ticket, &QPushButton::clicked, this, &MainWindow::addTicket);
    connect(ui->pushButton_delete_ticket, &QPushButton::clicked, this, &MainWindow::deleteTicket);
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

```
void MainWindow::addParking()
{
    QString id = ui->lineEdit_id_parking->text();
    QString gps = ui->lineEdit_gps->text();
    QString size = ui->lineEdit_size->text();
    QString type = ui->lineEdit_type->text();
    QString cost = ui->lineEdit_cost->text();

    // Получаем путь к каталогу приложения
    QString filePath = QCoreApplication::applicationDirPath() + "/parkings.txt";

    // Открываем файл для записи данных
    QFile file(filePath);
    if (file.open(QIODevice::Append | QIODevice::Text)) {
        QTextStream out(&file);
        out << id << ";" << gps << ";" << size << ";" << type << ";" << cost <<
";Имеется в наличии\n";
        file.close();

        qDebug() << "Данные сохранены в файл:" << filePath;
    } else {
        qDebug() << "Ошибка при открытии файла для записи";
        return;
    }

    // Очищаем таблицу перед загрузкой новых данных
    ui->tableParking->clearContents();
```

```
    ui->tableParking->setRowCount(0);

    // Получаем данные о парковках из файла и загружаем их в таблицу
    loadParkingsToTable();

    qDebug() << "Добавлена новая парковка:";
    qDebug() << "ID:" << id << "GPS:" << gps << "Размер:" << size << "Тип:" <<
type << "Оплата:" << cost;
}

void MainWindow::deleteParking()
{
    QString id = ui->lineEdit_id_parking->text();

    // Открываем файл для чтения данных
    QString filePath = QCoreApplication::applicationDirPath() + "/parkings.txt";
    QFile file(filePath);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }

    QTextStream in(&file);
    QString updatedData; // Создаем строку для хранения обновленных данных

    // Перебираем строки файла
    while (!in.atEnd()) {
        QString line = in.readLine();
```

```
        QStringList infoList = line.split(";");
        if (!infoList.isEmpty() && infoList.first() == id) {
            // Проверяем статус парковки
            if (infoList.last().trimmed() == "Занято") {
                QMessageBox::warning(this, "Ошибка", "Невозможно удалить
занятую парковку");
                file.close();
                return;
            }
        } else {
            // Добавляем строку в обновленные данные
            updatedData += line + "\n";
        }
    }

    file.close();

    // Открываем файл для записи данных
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text | QIODevice::Truncate))
{
        qDebug() << "Ошибка при открытии файла для записи";
        return;
    }

    QTextStream out(&file);

    // Перезаписываем файл с учетом изменений
    out << updatedData;
```

```
    file.close();

    // Удаляем парковку из таблицы
    for (int row = 0; row < ui->tableParking->rowCount(); ++row) {
        QTableWidgetItem *item = ui->tableParking->item(row, 0);
        if (item && item->text() == id) {
            ui->tableParking->removeRow(row);
            qDebug() << "Парковка успешно удалена";
            break;
        }
    }
}

void MainWindow::updateTable()
{
    loadParkingsToTable(); // Перезагружаем таблицу после добавления новой парковки
}

void MainWindow::loadParkingsToTable()
{
    // Очищаем таблицу перед загрузкой новых данных
    ui->tableParking->clearContents();
    ui->tableParking->setRowCount(0);

    // Устанавливаем количество колонок в таблице
    int numColumns = 6; // Для шести параметров
    ui->tableParking->setColumnCount(numColumns);
```

```
    // Устанавливаем заголовки колонок
    QStringList columnHeaders = {"ID", "Местоположение", "Размер", "Тип", "Оплата", "Наличие"};
    ui->tableParking->setHorizontalHeaderLabels(columnHeaders);

    // Получаем путь к файлу
    QString filePath = QCoreApplication::applicationDirPath() + "/parkings.txt";

    // Открываем файл для чтения данных
    QFile file(filePath);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }

    // Сет для хранения уникальных строк
    QSet<QString> uniqueRows;
    QTextStream in(&file);

    // Проходим по файлу и добавляем уникальные строки в таблицу и сет
    while (!in.atEnd()) {
        QString line = in.readLine();
        QStringList infoList = line.split(";");
        QString rowData = infoList.join(";"); // Склеиваем строки, чтобы проверять их на уникальность
        if (!uniqueRows.contains(rowData)) {
            uniqueRows.insert(rowData); // Добавляем строку в сет уникальных строк
```

```
            int row = ui->tableParking->rowCount();
            ui->tableParking->insertRow(row);
            for (int i = 0; i < infoList.size(); ++i) {
                QTableWidgetItem *item = new QTableWidgetItem(infoList.at(i));
                ui->tableParking->setItem(row, i, item);
            }
        }
    }

    file.close();

    // Перезаписываем файл, используя уникальные строки
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text | QIODevice::Truncate))
{
        qDebug() << "Ошибка при открытии файла для записи";
        return;
    }

    QTextStream out(&file);
    for (const QString &row : uniqueRows) {
        out << row << "\n"; // Пишем строку в файл
    }

    file.close();
}

void MainWindow::addVisitors()
{
```

```
    QString number = ui->lineEdit_visitors_number->text();
    QString name = ui->lineEdit_visitors_name->text();
    QString address = ui->lineEdit_visitors_adress->text();
    QString phone = ui->lineEdit_visitors_phone->text();

    visitorsFactory->addVisitor(number, name, address, phone);
    visitorsFactory->saveVisitorsToFile(); // Сохраняем посетителей в файл
}

void MainWindow::deleteVisitors()
{
    QString number = ui->lineEdit_visitors_number->text();

    // Открываем файл с билетами для чтения
    QString ticketFilePath = QCoreApplication::applicationDirPath() + "/ticket.txt";
    QFile ticketFile(ticketFilePath);
    if (!ticketFile.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла с билетами для чтения";
        return;
    }

    QTextStream ticketIn(&ticketFile);

    // Проверяем, есть ли билеты на данного посетителя
    bool hasTickets = false;
    while (!ticketIn.atEnd()) {
        QString line = ticketIn.readLine();
        QStringList infoList = line.split(";");
```

```
        if (!infoList.isEmpty() && infoList.at(1) == number) {
            hasTickets = true;
            break;
        }
    }

    ticketFile.close();

    // Если есть билеты на посетителя, выводим сообщение об ошибке и завершаем функцию
    if (hasTickets) {
        QMessageBox::warning(this, "Ошибка", "Невозможно удалить посетителя, на которого есть билеты");
        return;
    }

    // Создаем множество для хранения уникальных строк данных
    QSet<QString> uniqueRows;

    // Открываем файл для чтения данных о посетителях
    QString filePath = QCoreApplication::applicationDirPath() + "/visitors.txt";
    QFile file(filePath);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }

    QTextStream in(&file);
```

```
    // Заполняем множество уникальных строк данными из файла, исключая строки с данным номером посетителя
    while (!in.atEnd()) {
        QString line = in.readLine();
        QStringList infoList = line.split(";");
        if (infoList.isEmpty() || infoList.at(0) != number) {
            uniqueRows.insert(line);
        }
    }

    file.close();

    // Открываем файл для записи данных
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text | QIODevice::Truncate)) {
        qDebug() << "Ошибка при открытии файла для записи";
        return;
    }

    QTextStream out(&file);

    // Перезаписываем файл с учетом изменений
    foreach (const QString &row, uniqueRows) {
        out << row << "\n";
    }

    file.close();

    // Удаляем посетителя из таблицы
```

```
    for (int row = 0; row < ui->tableVisitors->rowCount(); ++row) {
        QTableWidgetItem *item = ui->tableVisitors->item(row, 0);
        if (item && item->text() == number) {
            ui->tableVisitors->removeRow(row);
            qDebug() << "Посетитель успешно удален";
            break;
        }
    }
}

void MainWindow::updateVisitorsTable()
{
    visitorsFactory->loadVisitorsFromfile(ui->tableVisitors);        //        Загружаем
посетителей из файла в таблицу
}

void MainWindow::addTicket()
{
    QString parkingId = ui->lineEdit_parking_number->text();
    QString visitorNumber = ui->lineEdit_visitors_number_t->text();
    QString date = ui->lineEdit_date->text();

    ticketManager->addTicket(parkingId, visitorNumber, date, ui->tableTicket);
    updateTicketTable();
    updateTable();
}

void MainWindow::deleteTicket()
```

```
{
    // Получаем ID парковки и номер посетителя, введенные пользователем
    QString parkingId = ui->lineEdit_parking_number->text();
    QString visitorNumber = ui->lineEdit_visitors_number_t->text();

    // Открываем файл с билетами для чтения и записи
    QString filePath = QCoreApplication::applicationDirPath() + "/ticket.txt";
    QFile file(filePath);
    if (!file.open(QIODevice::ReadWrite | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }

    // Считываем билеты из файла и удаляем выбранный билет
    QTextStream in(&file);
    QStringList ticketLines;
    while (!in.atEnd()) {
        QString line = in.readLine();
        QStringList infoList = line.split(";");
        if (infoList.size() >= 2 && infoList[0] == parkingId && infoList[1] ==
visitorNumber) {
            // Пропускаем удаление выбранного билета
            continue;
        }
        ticketLines.append(line);
    }
    file.resize(0); // Очищаем содержимое файла
    QTextStream out(&file);
```

```
    for (const QString &line : ticketLines) {
        out << line << "\n"; // Записываем оставшиеся билеты обратно в файл
    }
    file.close();

    // Обновляем таблицу билетов
    updateTicketTable();

    // Изменяем статус парковки, если билет возвращается
    QFile parkingFile(QCoreApplication::applicationDirPath() + "/parkings.txt");
    if (!parkingFile.open(QIODevice::ReadWrite | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }

    QTextStream parkingIn(&parkingFile);
    QString updatedParkingData;
    while (!parkingIn.atEnd()) {
        QString line = parkingIn.readLine();
        QStringList infoList = line.split(";");
        if (infoList.size() >= 1 && infoList[0] == parkingId &&
infoList.last().trimmed() == "Занято") {
            // Парковка становится доступной после возврата билета
            infoList.last() = "Имеется в наличии";
            line = infoList.join(";");
        }
        updatedParkingData += line + "\n";
    }
```

```
    parkingFile.resize(0); // Очищаем содержимое файла
    QTextStream parkingOut(&parkingFile);
    parkingOut << updatedParkingData; // Записываем обновленные данные о парковках обратно в файл
    parkingFile.close();
    updateTable();
}

void MainWindow::updateTicketTable()
{
    // Очищаем таблицу перед загрузкой новых данных
    ui->tableTicket->clearContents();
    ui->tableTicket->setRowCount(0);

    // Устанавливаем количество колонок в таблице
    int numColumns = 3; // Для трех параметров: ID парковки, Номер посетителя, Дата выдачи
    ui->tableTicket->setColumnCount(numColumns);

    // Устанавливаем заголовки колонок
    QStringList columnHeaders = {"ID парковки", "Номер посетителя", "Дата выдачи"};
    ui->tableTicket->setHorizontalHeaderLabels(columnHeaders);

    // Получаем путь к файлу
    QString filePath = QCoreApplication::applicationDirPath() + "/ticket.txt";

    // Открываем файл для чтения данных
    QFile file(filePath);
```

```
    if (!file.open(QIODevice::ReadWrite | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }

    QTextStream in(&file);
    QSet<QString> uniqueParkingIds; // Множество для хранения уникальных ID парковки
    QSet<QString> uniqueVisitorNumbers; // Множество для хранения уникальных номеров посетителя
    QStringList ticketLines; // Список строк для хранения записей без дубликатов

    // Проходим по файлу и добавляем строки в таблицу, проверяя на дубликаты только по ID парковки и номеру посетителя
    while (!in.atEnd()) {
        QString line = in.readLine();
        QStringList infoList = line.split(";");
        QString parkingId = infoList.at(0);
        QString visitorNumber = infoList.at(1);
        if (!uniqueParkingIds.contains(parkingId) && !uniqueVisitorNumbers.contains(visitorNumber)) {
            uniqueParkingIds.insert(parkingId); // Добавляем уникальный ID парковки в множество
            uniqueVisitorNumbers.insert(visitorNumber); // Добавляем уникальный номер посетителя в множество
            ticketLines.append(line); // Добавляем строку без дубликатов в список
            int row = ui->tableTicket->rowCount();
            ui->tableTicket->insertRow(row);
            for (int i = 0; i < infoList.size(); ++i) {
                QTableWidgetItem *item = new QTableWidgetItem(infoList.at(i));
```

```
            ui->tableTicket->setItem(row, i, item);
        }
    }
}

file.resize(0); // Очищаем содержимое файла
QTextStream out(&file);
for (const QString &line : ticketLines) {
    out << line << "\n"; // Записываем строки без дубликатов обратно в файл
}

file.close();
}
```

parkingfactory.cpp

```
#include "parkingfactory.h"
#include <QTableWidgetItem>

ParkingFactory::ParkingFactory(QObject *parent) : QObject(parent) {}

ParkingFactory::~ParkingFactory() {}

void ParkingFactory::addParking(const QString &id, const QString &gps, const QString &size, const QString &type, const QString &cost)
{
    // Формируем строку данных о парковке
    QString parkingInfo = id + ";" + gps + ";" + size + ";" + type + ";" + cost + ";Имеется в наличии";
    parkingData.append(parkingInfo);
```

```
    qDebug() << "Добавлена новая парковка:";
    qDebug() << "ID:" << id << "GPS:" << gps << "Размер:" << size << "Тип:" << type << "Оплата:" << cost;

    // Отправляем сигнал о добавлении новой парковки
    emit parkingAdded();
}

void ParkingFactory::loadParkingsToTable(QTableWidget *table)
{
    // Очищаем таблицу перед загрузкой новых данных
    table->clearContents();
    table->setRowCount(0);

    // Проходим по списку данных о парковках и добавляем их в таблицу
    for (const QString &parkingInfo : parkingData) {
        QStringList infoList = parkingInfo.split(";");
        int row = table->rowCount();
        table->insertRow(row);
        for (int i = 0; i < infoList.size(); ++i) {
            QTableWidgetItem *item = new QTableWidgetItem(infoList.at(i));
            table->setItem(row, i, item);
        }
    }
}
```

ticketmanager.cpp

```
#include "ticketmanager.h"
#include <QCoreApplication>

TicketManager::TicketManager(QObject *parent) : QObject(parent)
{

}

void TicketManager::addTicket(const QString &parkingId, const QString &visitorNumber, const QString &date, QTableWidget *table)
{
    // Проверяем, существует ли парковка с указанным ID
    QString parkingFilePath = QCoreApplication::applicationDirPath() + "/parkings.txt";
    QFile parkingFile(parkingFilePath);
    if (!parkingFile.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }

    QTextStream parkingIn(&parkingFile);
    bool parkingExists = false;
    while (!parkingIn.atEnd()) {
        QString line = parkingIn.readLine();
        QStringList infoList = line.split(";");
        if (!infoList.isEmpty() && infoList.first() == parkingId) {
            parkingExists = true;
            break;
        }
```

```
    }
    parkingFile.close();

    if (!parkingExists) {
        QMessageBox::warning(nullptr, "Ошибка", "Парковка с указанным ID не найдена!");
        return;
    }

    // Проверяем, существует ли посетитель с указанным номером
    QString visitorsFilePath = QCoreApplication::applicationDirPath() + "/visitors.txt";
    QFile visitorsFile(visitorsFilePath);
    if (!visitorsFile.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }

    QTextStream visitorsIn(&visitorsFile);
    bool visitorExists = false;
    while (!visitorsIn.atEnd()) {
        QString line = visitorsIn.readLine();
        QStringList infoList = line.split(";");
        if (!infoList.isEmpty() && infoList.first() == visitorNumber) {
            visitorExists = true;
            break;
        }
    }
    visitorsFile.close();
```

```
    if (!visitorExists) {
        QMessageBox::warning(nullptr, "Ошибка", "Посетитель с указанным номером не найден!");
        return;
    }

    // Создаем билет
    QString ticketFilePath = QCoreApplication::applicationDirPath() + "/ticket.txt";
    QFile ticketFile(ticketFilePath);
    if (!ticketFile.open(QIODevice::Append | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для записи";
        return;
    }

    QTextStream out(&ticketFile);
    out << parkingId << ";" << visitorNumber << ";" << date << "\n";
    ticketFile.close();

    // Обновляем таблицу билетов
    updateTicketTable(table);

    // Изменяем статус парковки
    QFile parkingFileModify(parkingFilePath);
    if (!parkingFileModify.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }
```

```
    QString updatedParkingData;
    QTextStream parkingInModify(&parkingFileModify);
    while (!parkingInModify.atEnd()) {
        QString line = parkingInModify.readLine();
        QStringList infoList = line.split(";");
        if (!infoList.isEmpty() && infoList.first() == parkingId) {
            infoList.last() = "Занято";
            line = infoList.join(";");
        }
        updatedParkingData += line + "\n";
    }
    parkingFileModify.close();

    if (!parkingFileModify.open(QIODevice::WriteOnly | QIODevice::Text |
QIODevice::Truncate)) {
        qDebug() << "Ошибка при открытии файла для записи";
        return;
    }

    QTextStream parkingOutModify(&parkingFileModify);
    parkingOutModify << updatedParkingData;
    parkingFileModify.close();
}

void TicketManager::updateTicketTable(QTableWidget *table)
{
    // Очищаем таблицу перед загрузкой новых данных
```

```
    table->clearContents();
    table->setRowCount(0);

    // Устанавливаем количество колонок в таблице
    int numColumns = 3; // Для трех параметров
    table->setColumnCount(numColumns);

    // Устанавливаем заголовки колонок
    QStringList columnHeaders = {"ID парковки", "Номер посетителя", "Дата выдачи"};
    table->setHorizontalHeaderLabels(columnHeaders);

    // Получаем путь к файлу
    QString filePath = QCoreApplication::applicationDirPath() + "/ticket.txt";

    // Открываем файл для чтения данных
    QFile file(filePath);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }

    QTextStream in(&file);

    // Проходим по файлу и добавляем строки в таблицу
    while (!in.atEnd()) {
        QString line = in.readLine();
        QStringList infoList = line.split(";");
```

```
        int row = table->rowCount();
        table->insertRow(row);
        for (int i = 0; i < infoList.size(); ++i) {
            QTableWidgetItem *item = new QTableWidgetItem(infoList.at(i));
            table->setItem(row, i, item);
        }
    }

    file.close();
}
```

visitors.cpp

```
#include "visitors.h"
#include <QCoreApplication>
#include <QApplication>

Visitors::Visitors(QObject *parent) : QObject(parent) {}

void Visitors::addVisitor(const QString &number, const QString &name, const QString &address, const QString &phone)
{
    QStringList visitorInfo = {number, name, address, phone};
    visitorData.append(visitorInfo);

    emit visitorsAdded();
}

void Visitors::saveVisitorsToFile()
```

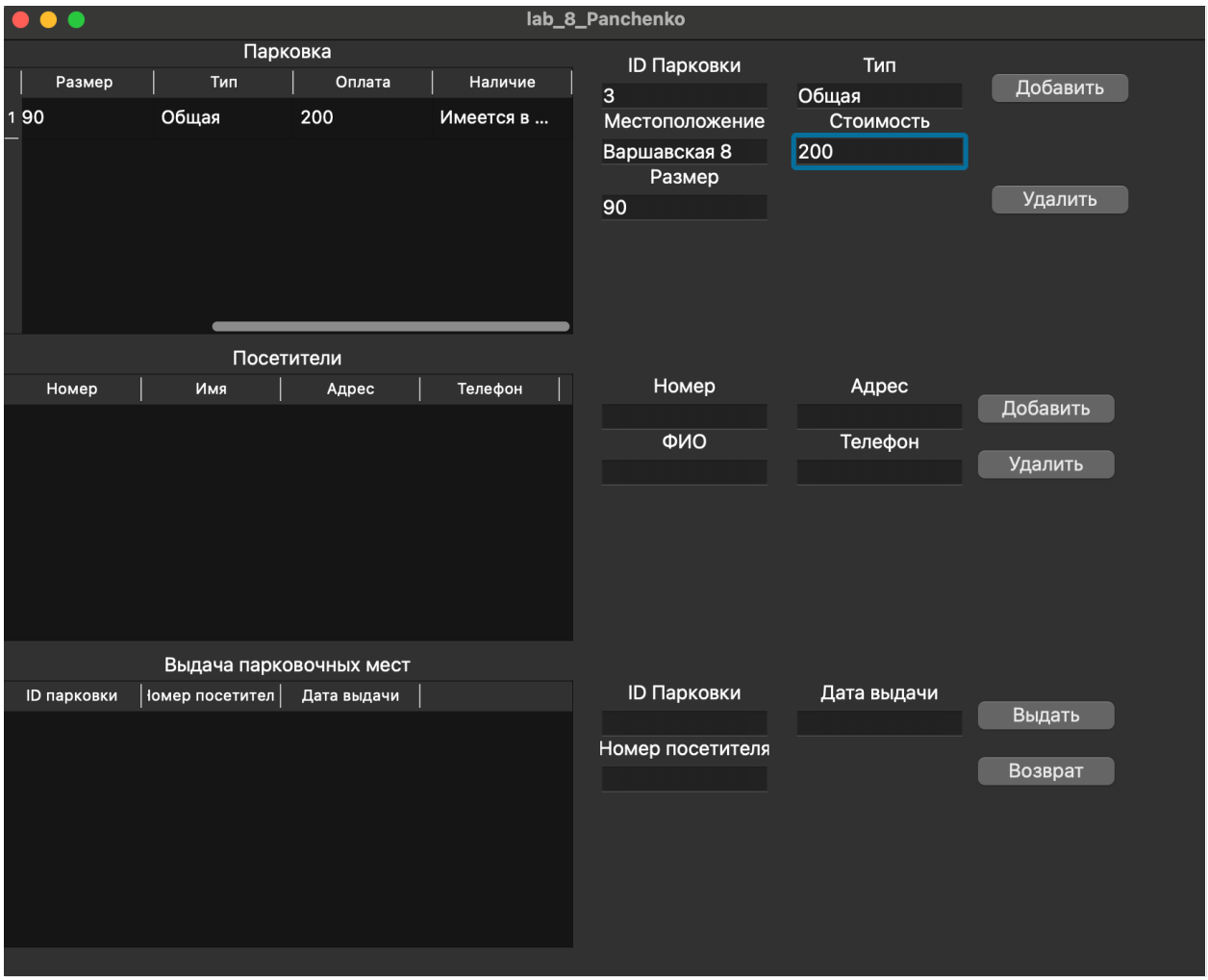
```
{
    QString filePath = QCoreApplication::applicationDirPath() + "/visitors.txt";
    QFile file(filePath);
    if (file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        QTextStream out(&file);
        QSet<QString> uniqueRows; // Сет для хранения уникальных строк
        for (const QStringList &visitorInfo : visitorData) {
            QString rowData = visitorInfo.join(";"); // Преобразуем список в строку
            if (!uniqueRows.contains(rowData)) {
                uniqueRows.insert(rowData); // Добавляем строку в сет уникальных
строк
                out << rowData << "\n"; // Записываем уникальную строку в файл
            }
        }
        file.close();
    }
}

void Visitors::loadVisitorsFromfile(QTableWidget *table)
{
    // Очищаем таблицу перед загрузкой новых данных
    table->clearContents();
    table->setRowCount(0);

    // Устанавливаем количество колонок в таблице
    int numColumns = 4; // Для четырех параметров
    table->setColumnCount(numColumns);
```

```
    // Устанавливаем заголовки колонок
    QStringList columnHeaders = {"Номер", "Имя", "Адрес", "Телефон"};
    table->setHorizontalHeaderLabels(columnHeaders);

    // Получаем путь к файлу
    QString filePath = QCoreApplication::applicationDirPath() + "/visitors.txt";

    // Открываем файл для чтения данных
    QFile file(filePath);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "Ошибка при открытии файла для чтения";
        return;
    }

    // Сет для хранения уникальных строк
    QSet<QString> uniqueRows;
    QTextStream in(&file);
    QString fileContent; // Строка для хранения содержимого файла

    // Проходим по файлу и добавляем уникальные строки в таблицу и сет
    while (!in.atEnd()) {
        QString line = in.readLine();
        QStringList infoList = line.split(";");
        QString rowData = infoList.join(";"); // Склеиваем строки, чтобы проверять
их на уникальность
        if (!uniqueRows.contains(rowData)) {
            uniqueRows.insert(rowData); // Добавляем строку в сет уникальных
строк
```

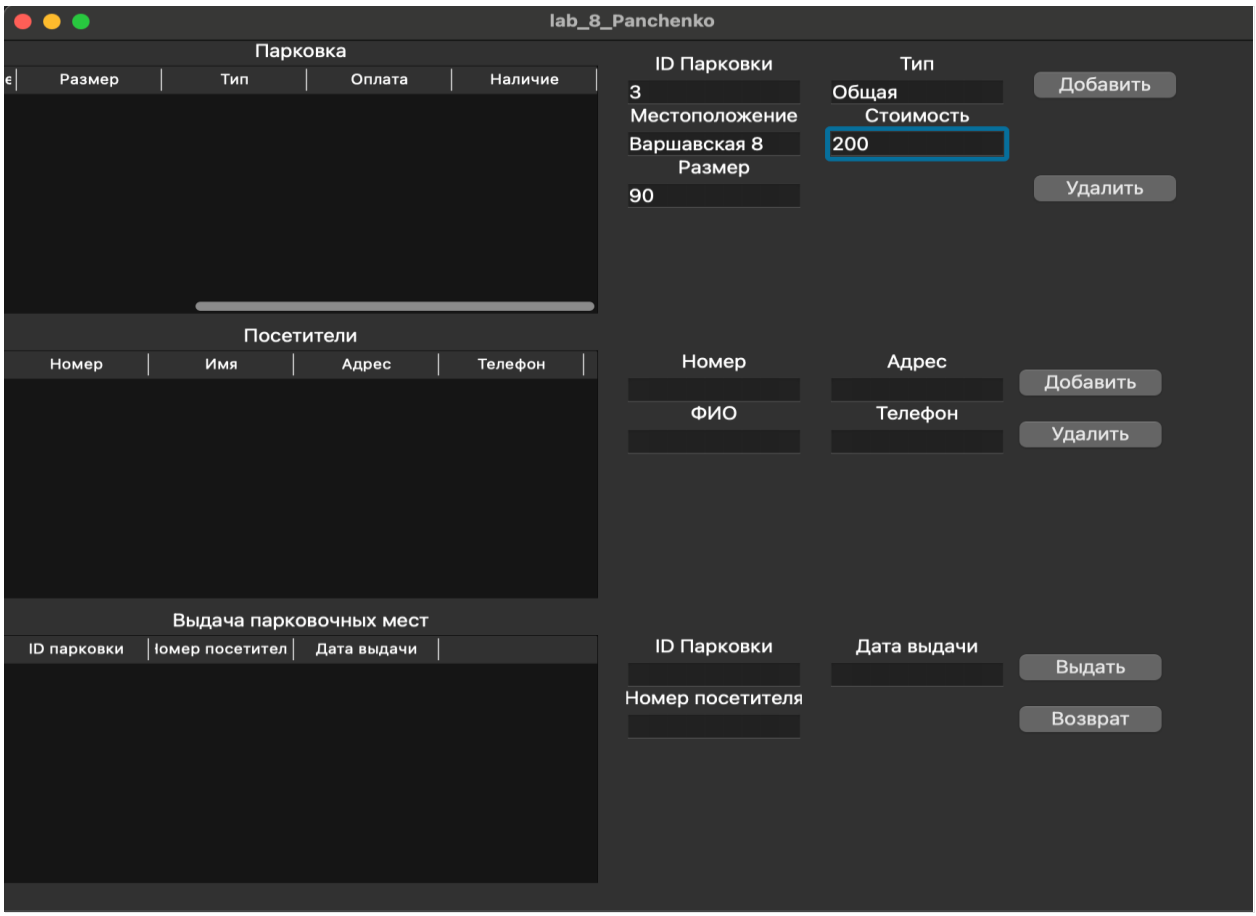
```
        fileContent.append(line + "\n"); // Добавляем строку к содержимому файла
        int row = table->rowCount();
        table->insertRow(row);
        for (int i = 0; i < infoList.size(); ++i) {
            QTableWidgetItem *item = new QTableWidgetItem(infoList.at(i));
            table->setItem(row, i, item);
        }
    }
}

file.close();

// Перезаписываем файл, используя уникальные строки
if (!file.open(QIODevice::WriteOnly | QIODevice::Text | QIODevice::Truncate)) {
    qDebug() << "Ошибка при открытии файла для записи";
    return;
}

QTextStream out(&file);
out << fileContent; // Пишем содержимое файла без дубликатов
file.close();
}
```

Скриншоты результатов работы

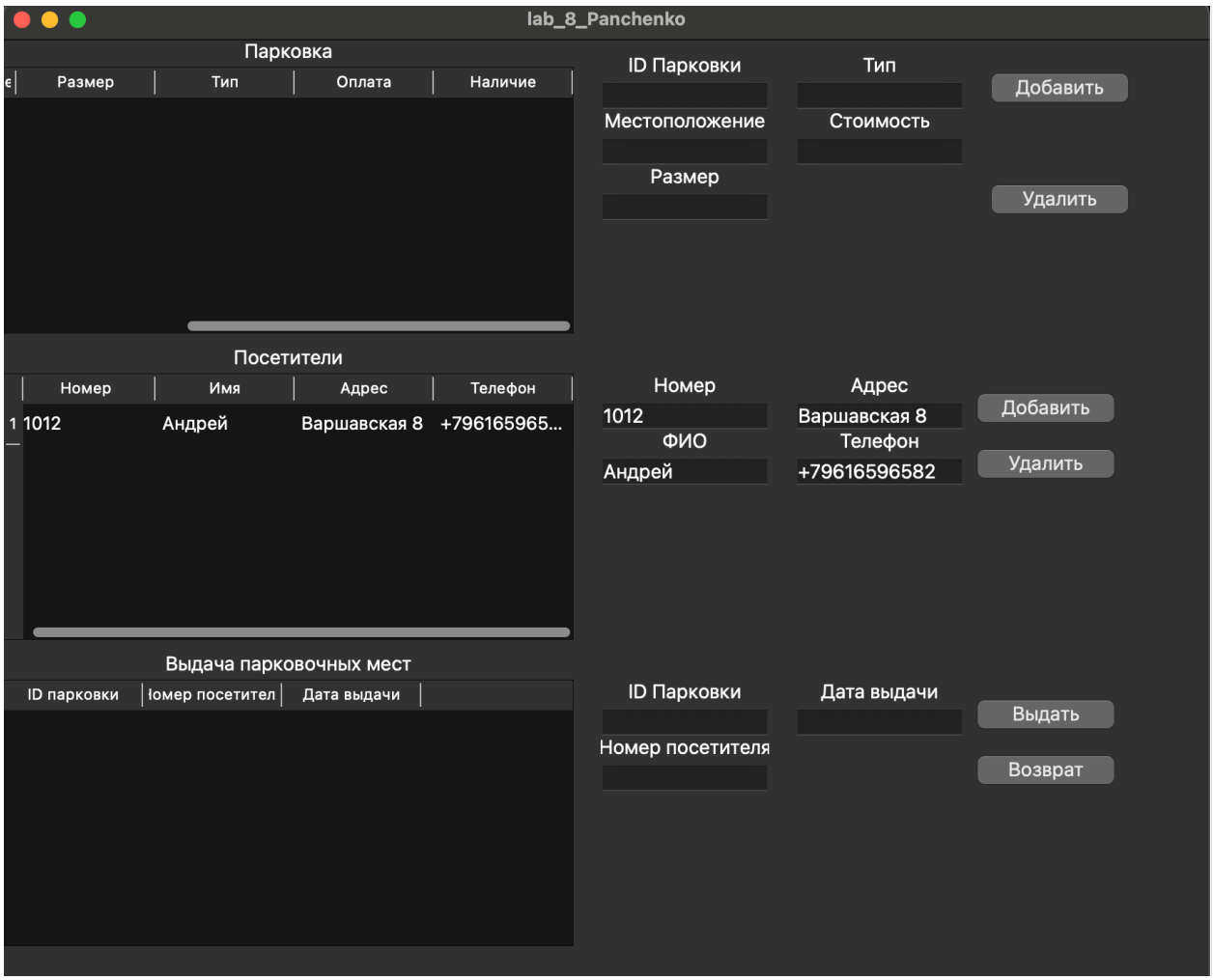
Добавляем данные в таблицу Парковка



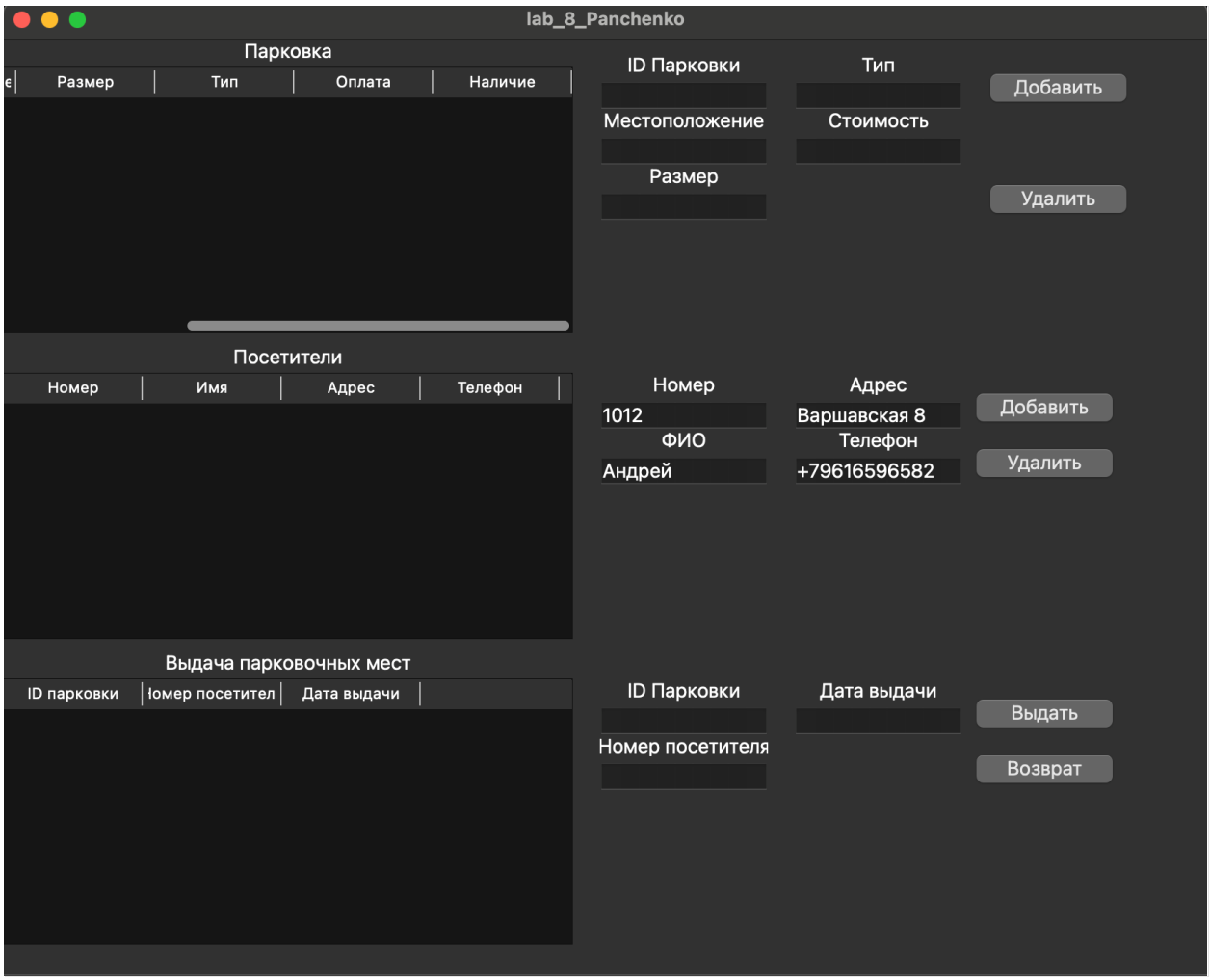
Удаляем данные из таблицы Парковка



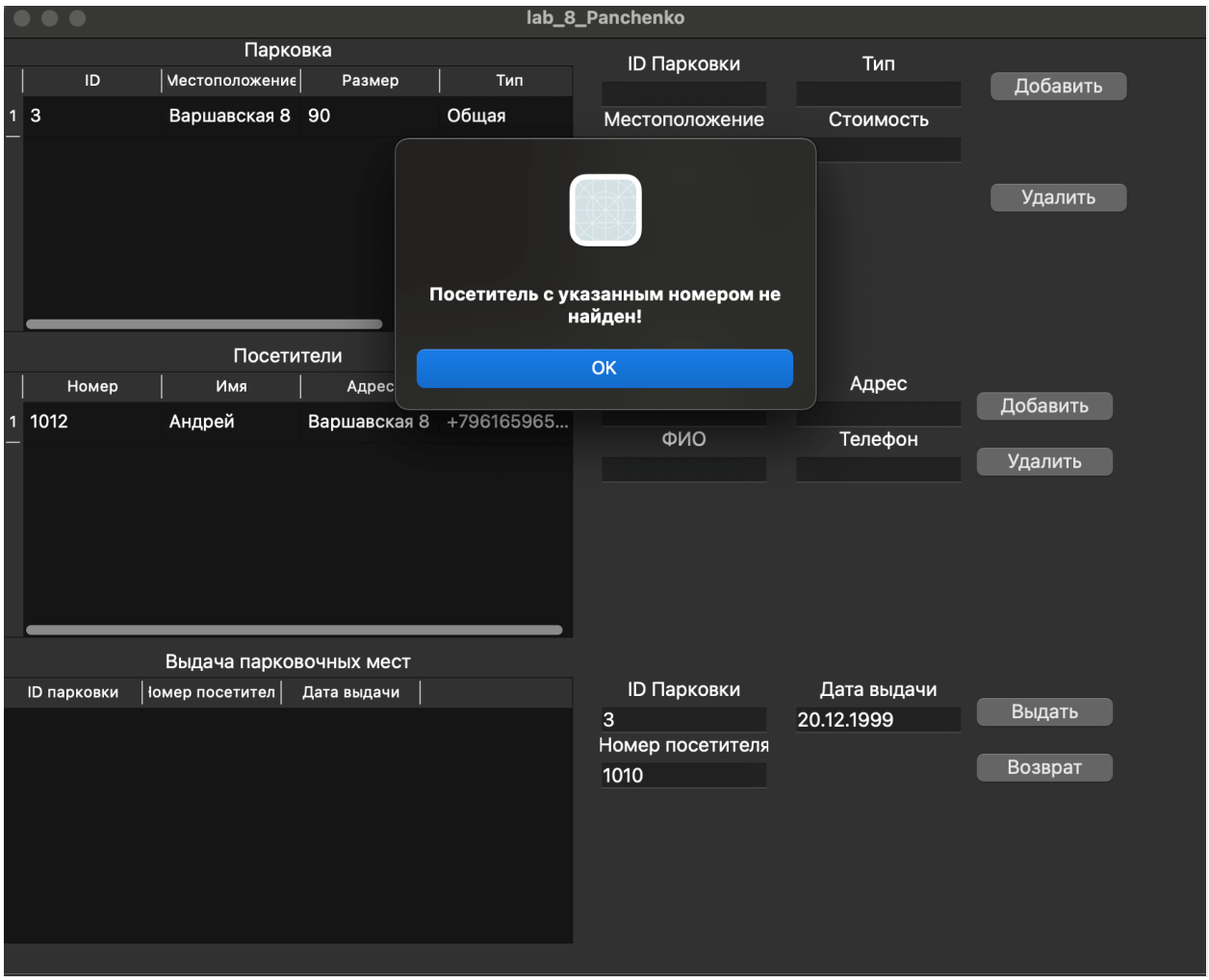
Добавляем данные в таблицу Посетители

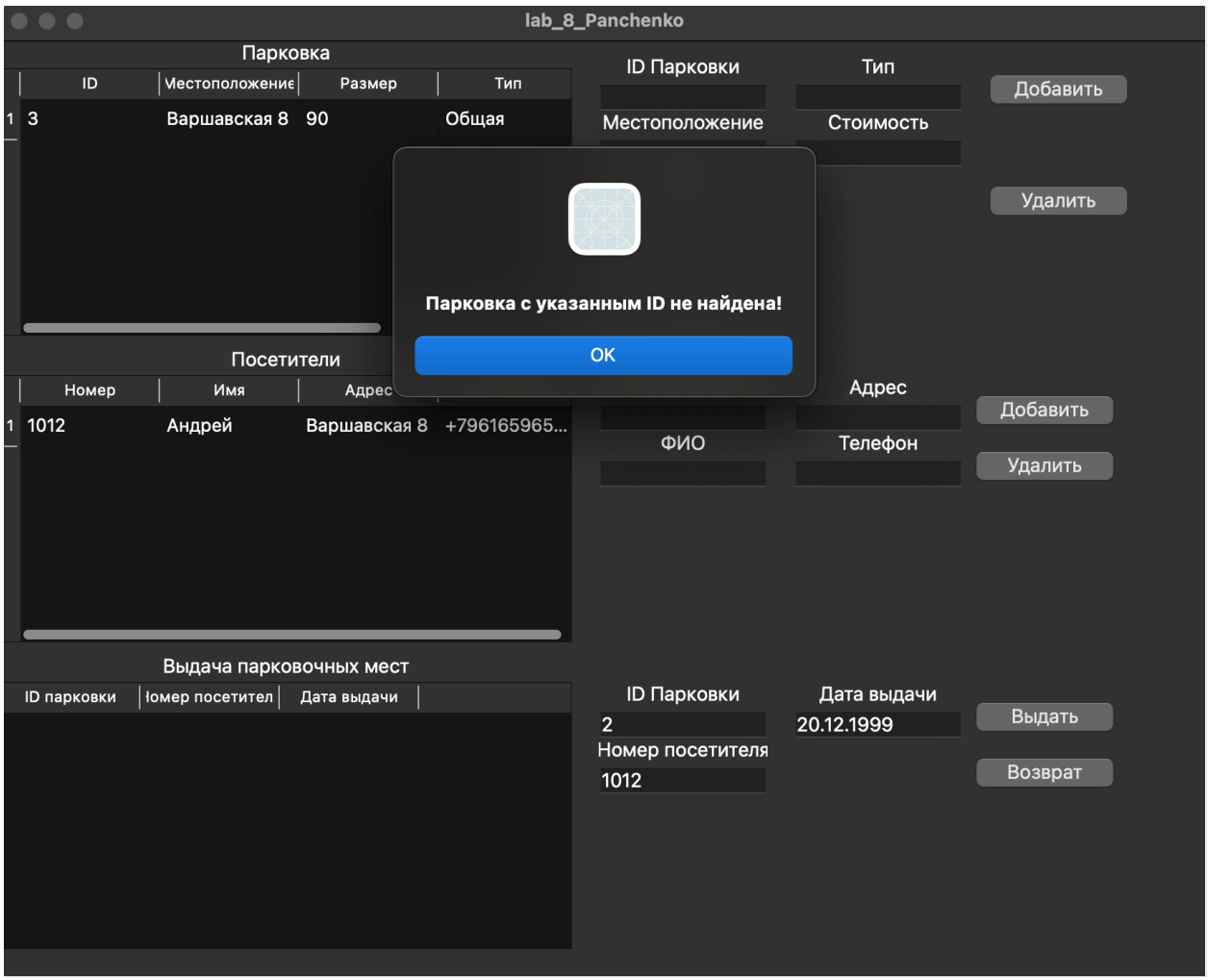


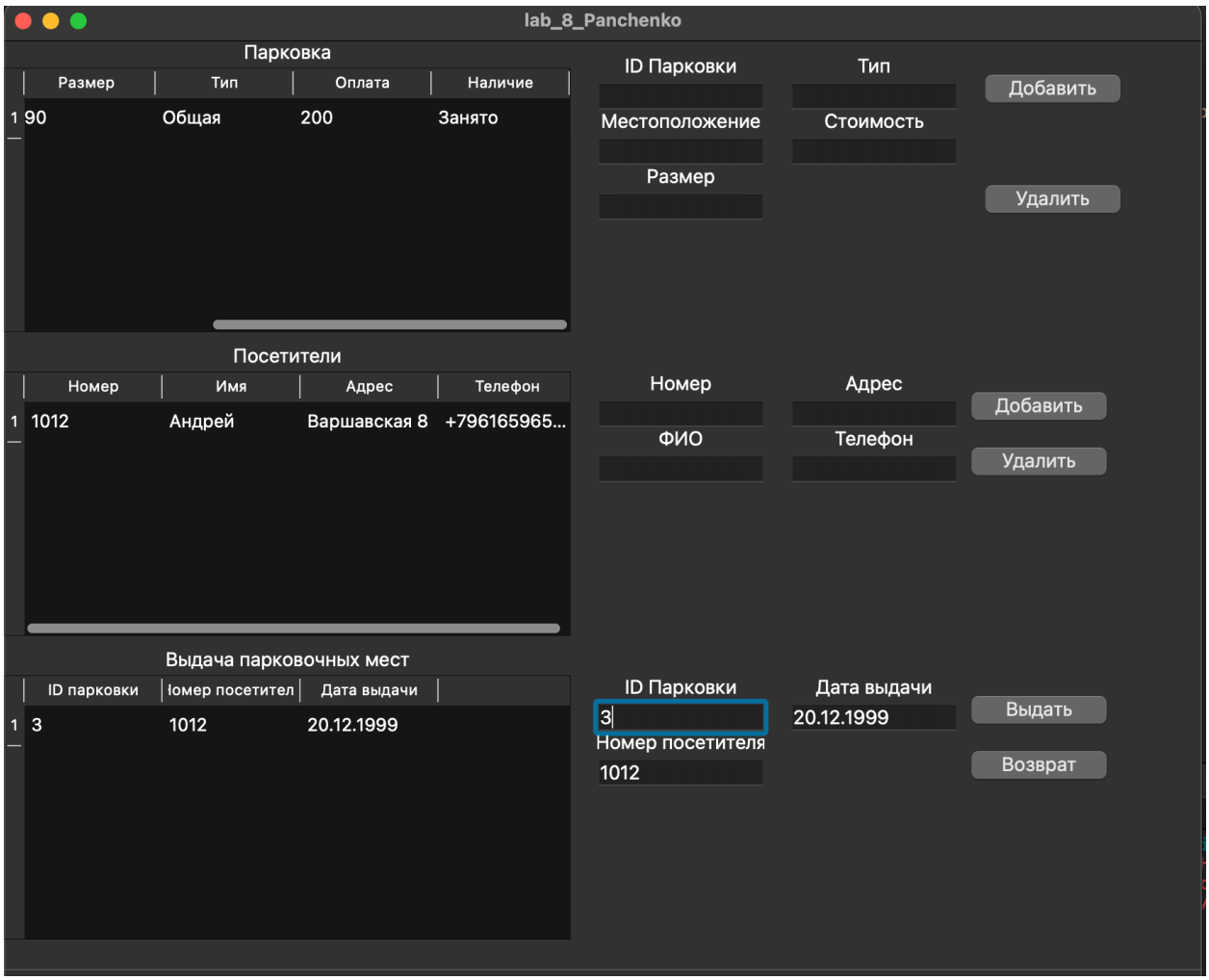
Удаляем данные из таблицы Посетители



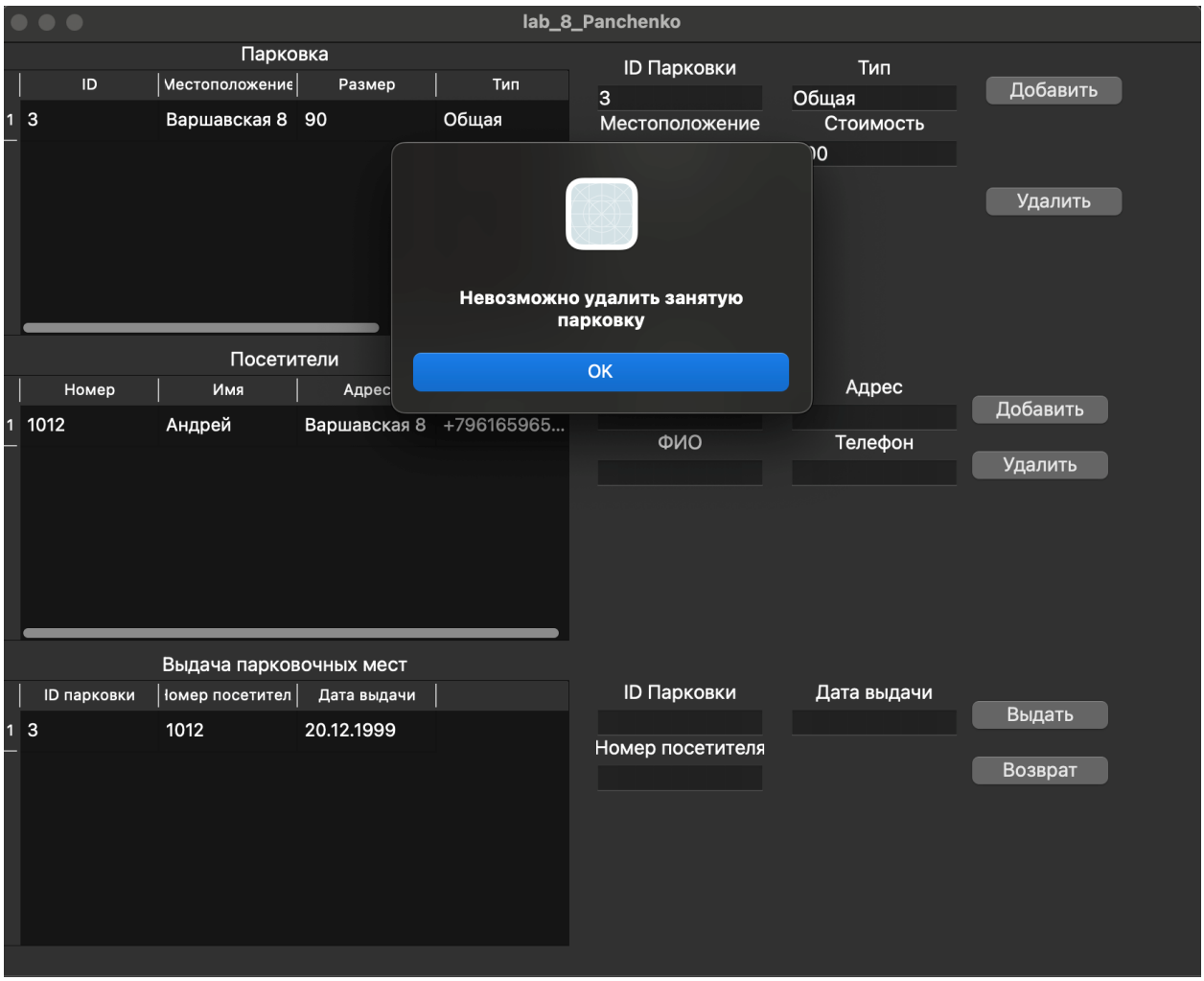
Выдаем билет посетителю на конкретную парковку если нету посетителя с таким номером или нет такой парковки, то высвечивается сообщение

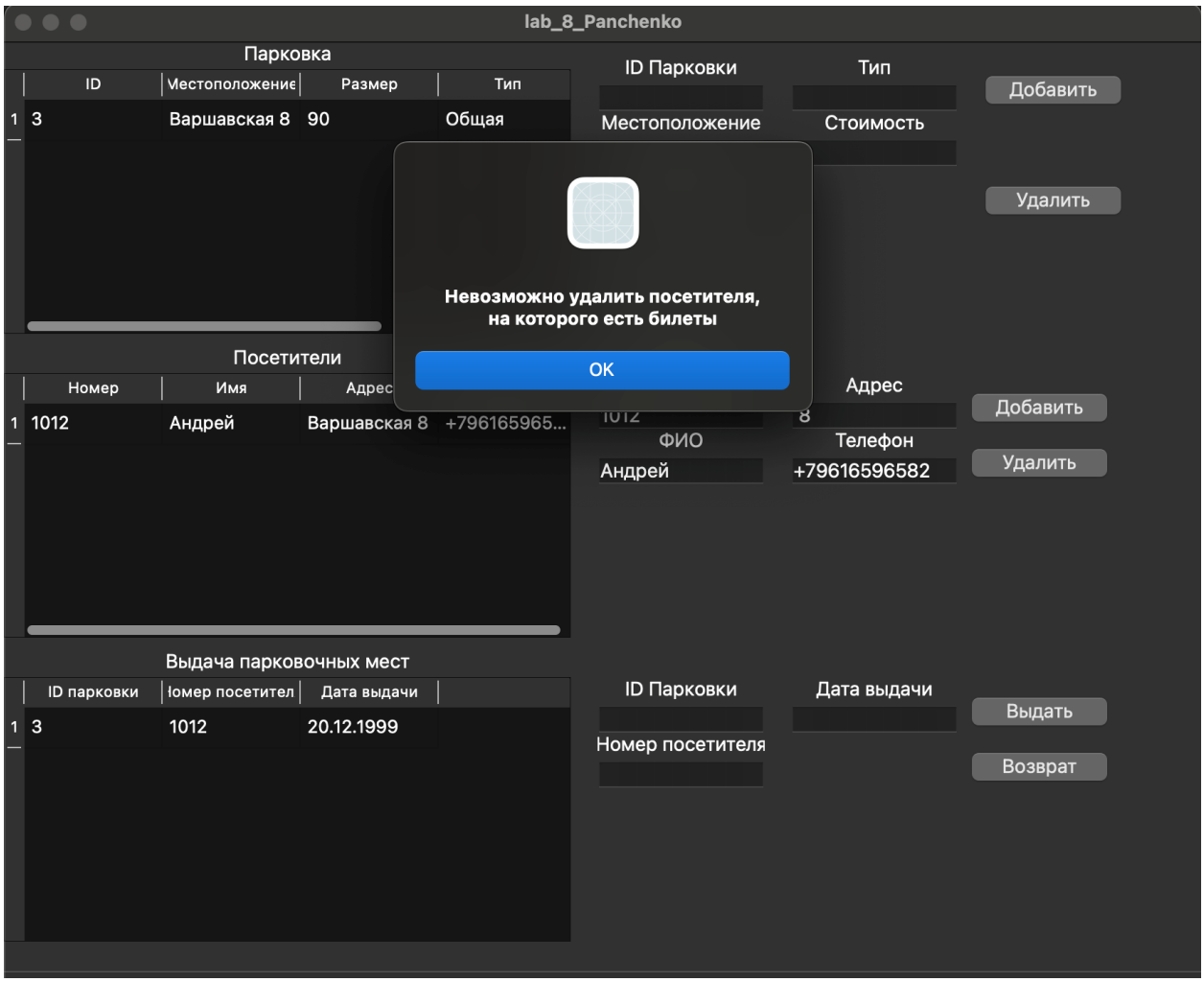

lab_8_Panchenko
Парковка
ID
Местоположение
Размер
Тип
1
3
Варшавская 8
90
Общая
ID Парковки
Тип
Добавить
Местоположение
Стоимость
Удалить
Посетитель с указанным номером не найден!
OK
Посетители
Номер
Имя
Адрес
1
1012
Андрей
Варшавская 8
+796165965...
Адрес
Добавить
ФИО
Телефон
Удалить
Выдача парковочных мест
ID парковки
Номер посетител
Дата выдачи
ID Парковки
Дата выдачи
3
20.12.1999
Выдать
Номер посетителя
1010
Возврат


lab_8_Panchenko
Парковка
ID
Местоположение
Размер
Тип
1
3
Варшавская 8
90
Общая
ID Парковки
Тип
Добавить
Местоположение
Стоимость
Удалить
Парковка с указанным ID не найдена!
OK
Посетители
Номер
Имя
Адрес
1
1012
Андрей
Варшавская 8
+796165965...
Адрес
Добавить
ФИО
Телефон
Удалить
Выдача парковочных мест
ID парковки
Номер посетител
Дата выдачи
ID Парковки
Дата выдачи
2
20.12.1999
Выдать
Номер посетителя
1012
Возврат

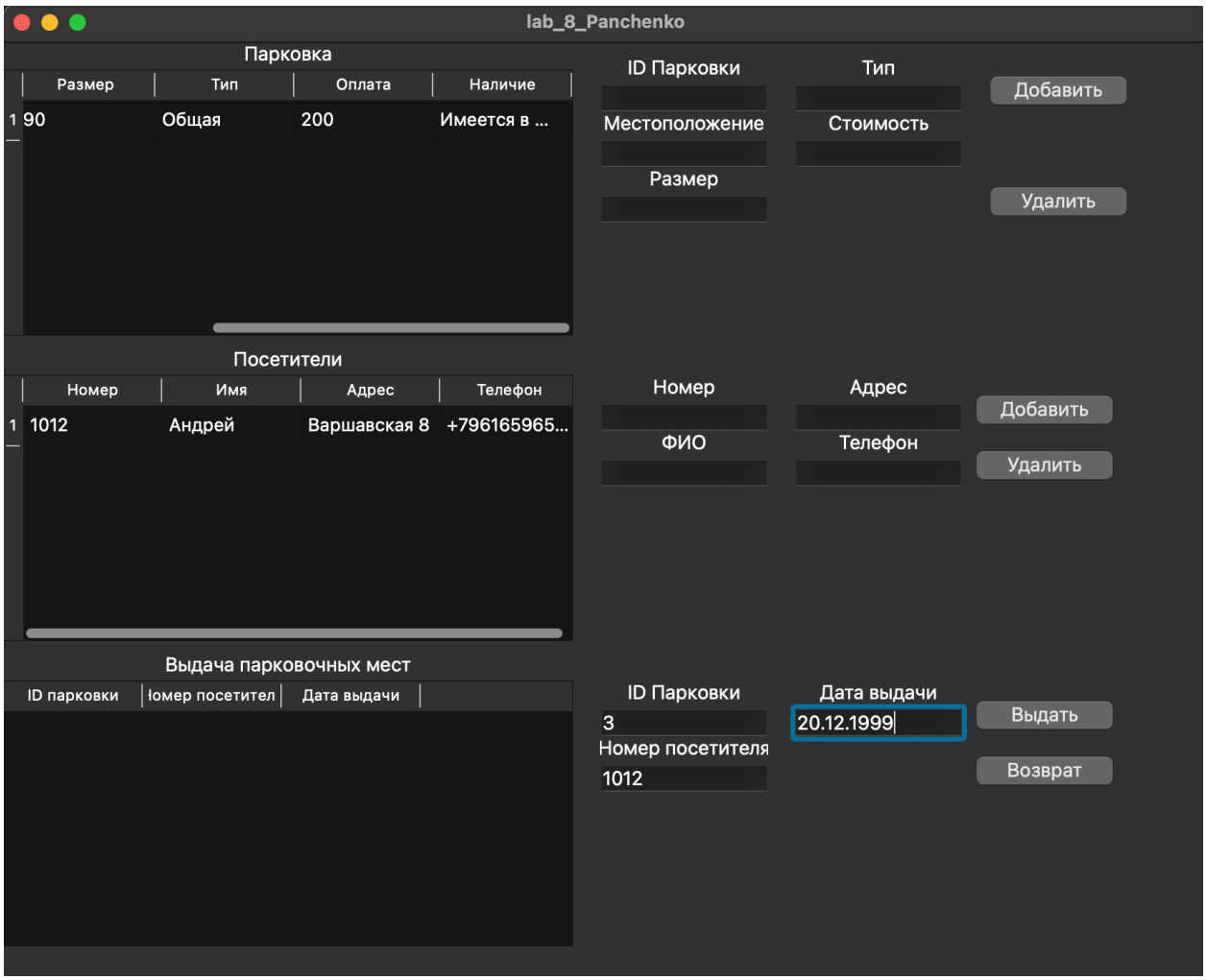


Удалить занятую парковку или посетителя у которого есть билет мы не можем


lab_8_Panchenko
Парковка
ID
Местоположение
Размер
Тип
1 3 Варшавская 8 90 Общая
ID Парковки
3
Тип
Общая
Добавить
Местоположение
Стоимость
Удалить
Невозможно удалить занятую парковку
OK
Посетители
Номер
Имя
Адрес
1 1012 Андрей Варшавская 8 +796165965...
Адрес
Добавить
ФИО
Телефон
Удалить
Выдача парковочных мест
ID парковки
Номер посетител
Дата выдачи
1 3 1012 20.12.1999
ID Парковки
Дата выдачи
Выдать
Номер посетителя
Возврат



Возврат билета


lab_8_Panchenko
Парковка
Размер
Тип
Оплата
Наличие
1 90
Общая
200
Имеется в ...
ID Парковки
Тип
Добавить
Местоположение
Стоимость
Размер
Удалить
Посетители
Номер
Имя
Адрес
Телефон
1 1012
Андрей
Варшавская 8
+796165965...
Номер
Адрес
Добавить
ФИО
Телефон
Удалить
Выдача парковочных мест
ID парковки
Номер посетител
Дата выдачи
ID Парковки
Дата выдачи
3
20.12.1999
Выдать
Номер посетителя
1012
Возврат

Вывод

В ходе разработки на Qt и использования Qt Creator я приобрел опыт создания графического интерфейса, управления различными виджетами и обработки событий через сигналы и слоты. В моем коде представлены различные виджеты, такие как QTableWidget, QLineEdit, QLabel и QPushButton, которые используются для создания пользовательского интерфейса для управления данными о парковке и посетителях.

Я также изучил работу с моделями данных для таблиц, что отражено в методах загрузки данных о парковках в QTableWidget. Это позволяет мне эффективно управлять данными и отображать их в графическом интерфейсе.

Мой код также демонстрирует понимание принципов работы паттернов проектирования, таких как фабричный метод. Например, у меня есть класс ParkingFactory, который используется для создания объектов парковок различных типов, обеспечивая гибкость и расширяемость приложения.

В общем, мой опыт работы с Qt позволил мне овладеть различными аспектами разработки приложений с графическим интерфейсом, обработки данных и применения паттернов проектирования для создания более эффективной и гибкой архитектуры приложения.